GUÍA PRÁCTICA Y ALGORITMOS DE GRAFOS

Estructuras de Datos, Manipulación de Memoria y Algoritmos de Examen (NASPOO)

Módulo 1: Consulta de Vecindades (getSucesores y getPredecesores)

En una representación por Lista de Adyacencia orientada a salidas, cada nodo vértice contiene una lista enlazada de sus arcos dirigidos salientes. Esta asimetría estructural determina directamente la complejidad de las dos consultas fundamentales de vecindad:

• **getSucesores(g, v)**: Acceso directo. Se localiza el vértice *v* en O(|V|) y se extraen los destinos de su lista de arcos salientes en O(grado■(v)).
• **getPredecesores(g, v)**: Barrido global. Requiere inspeccionar la lista de adyacencia de *todos* los vértices del grafo para determinar cuáles apuntan a *v*, con complejidad O(|V| + |E|).

Pseudocódigo NASPOO — getSucesores

```
func getSucesores(g: Grafo<T>, v: T) : Lista<T>
Var
    res: Lista<T>
    pVert: Pointer to NodoVertice<T>
    pArco: Pointer to NodoArco<T>
Begin
    res.construir()
    pVert <- g.primero

    while (pVert != NIL) and (pVert^.info != v) do
        pVert <- pVert^.siguiente
    EndWhile

    if pVert != NIL then
        pArco <- pVert^.listaAdg
        while pArco != NIL do
            res.insertar(pArco^.destino^.info, res.getLongitud() + 1)
            pArco <- pArco^.siguiente
        EndWhile
    endif

    return res
FFunc
```

Pseudocódigo NASPOO — getPredecesores

```
func getPredecesores(g: Grafo<T>, v: T) : Lista<T>
Var
    res: Lista<T>
    pVert: Pointer to NodoVertice<T>
    pArco: Pointer to NodoArco<T>
    esPredecesor: bool
Begin
    res.construir()
    pVert <- g.primero

    while pVert != NIL do
        esPredecesor <- false
        pArco <- pVert^.listaAdg

        while (pArco != NIL) and (!esPredecesor) do
            if pArco^.destino^.info = v then
                esPredecesor <- true
            else
                pArco <- pArco^.siguiente
            endif
        EndWhile

        if esPredecesor then
            res.insertar(pVert^.info, res.getLongitud() + 1)
        endif

        pVert <- pVert^.siguiente
    EndWhile

    return res
FFunc
```

Navegación de Punteros en Memoria

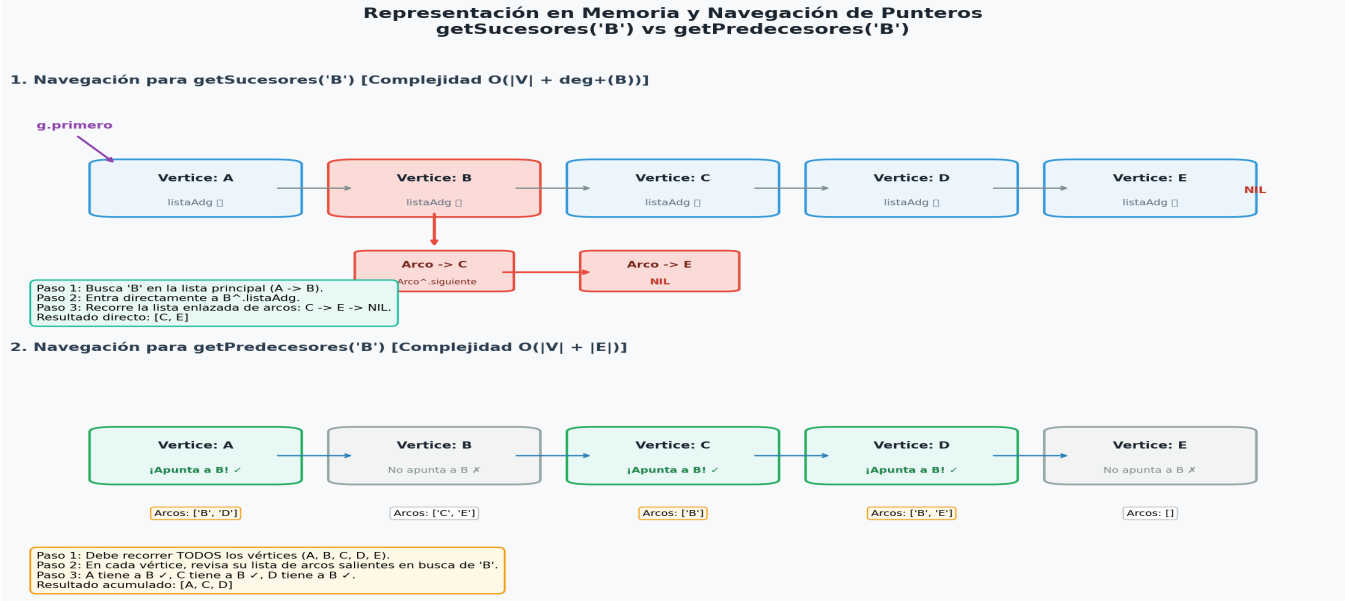


Figura 1: Representación del acceso vertical directo para getSucesores vs barrido horizontal global para getPredecesores.

Módulo 2: Modificación Estructural (eliminarVerticeSeguro)

La eliminación de un vértice en una estructura multienlazada exige desconectar previamente todos sus arcos incidentes (tanto entrantes como salientes). Si un nodo se remueve de la lista principal de vértices sin desvincular las aristas que otros nodos conservan hacia él, la estructura queda en un estado inconsistente con punteros colgantes (*dangling pointers*).

```
proc eliminarVerticeSeguro(ref g: Grafo<T>, v: T)
Var
    predecesores, sucesores: Lista<T>
    u, w: T
Begin
    if g.existeVertice(v) then
        // 1. Desconectar arcos entrantes (u -> v)
        predecesores <- g.getPredecesores(v)
        while !predecesores.esVacia() do
            u <- predecesores.consultar(1)
            g.eliminarArco(u, v)
            predecesores.eliminar(1)
        EndWhile

        // 2. Desconectar arcos salientes (v -> w)
        sucesores <- g.getSucesores(v)
        while !sucesores.esVacia() do
            w <- sucesores.consultar(1)
            g.eliminarArco(v, w)
            sucesores.eliminar(1)
        EndWhile

        // 3. Eliminar vértice completamente aislado
        g.eliminarVertice(v)
    endif
FProc
```

Secuencia de Aislamiento y Eliminación

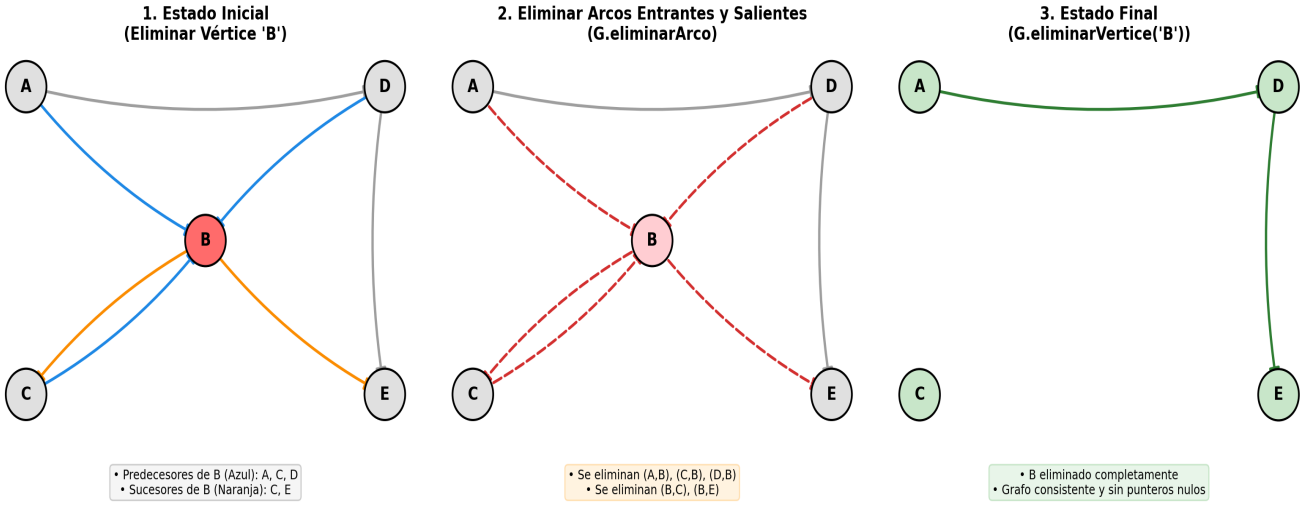


Figura 2: Proceso en tres fases: (1) Estado inicial, (2) Desconexión de aristas incidentes, (3) Remoción del nodo aislado.

Módulo 3: Exploración Exhaustiva y Backtracking (contarCaminosSimples)

Para contar o evaluar todos los caminos simples entre un nodo origen *v* y un destino *w*, el recorrido no puede detenerse al descubrir el primer camino. Se aplica la técnica de **Backtracking**: al regresar de una exploración recursiva, el estado del vértice examinado se restablece (visitados[u] <- false). Esto permite que ese mismo vértice sea reutilizado por rutas alternativas que provengan de otras ramas.

```
func contarCaminosSimples(g: Grafo<T>, v: T, w: T) : int
Var
    visitados: Array [1..N] of bool
    totalCaminos: int
    i: int
Begin
    for i <- 1 to g.getNVertices() do
        visitados[i] <- false
    EFor

    visitados[v] <- true
    totalCaminos <- 0
    contarDFSProc(g, v, w, visitados, totalCaminos)

    return totalCaminos
FFunc
proc contarDFSProc(g: Grafo<T>, actual: T, destino: T, ref visitados: Array of bool, ref total: int)
Var
    sucesores: Lista<T>
    u: int
Begin
    if actual = destino then
        total <- total + 1
    else
        sucesores <- g.getSucesores(actual)
        while !sucesores.esVacia() do
            u <- sucesores.consultar(1)

            if !visitados[u] then
                visitados[u] <- true
                contarDFSProc(g, u, destino, visitados, total)

                // BACKTRACKING: Desmarcar nodo para habilitar otras rutas
                visitados[u] <- false
            endif

            sucesores.eliminar(1)
        EndWhile
    endif
FProc
```

Corrida en Frío Guiada Paso a Paso (Grafo de A a D)

Paso	Acción Ejecutada	Pila Recursiva	Estado visitados	total (ref)	Explicación del Backtracking
1	Inicio en A, marca A	[A]	visitados['A']=true	0	Inicializa el wrapper e invoca contarDFSProc.

2	A llama a vecino B	[A, B]	visitados['A','B']=true	0	Entra recursivamente a B.
3	B llama a vecino D (DESTINO)	[A, B, D]	visitados['A','B','D']=true	1	Caso base: actual=destino. Incrementa total <- 0+1.
4	Termina D, regresa a B	[A, B]	visitados['D']=false	1	**BACKTRACKING:** Desmarca D para permitir otros caminos.
5	B llama a vecino C	[A, B, C]	visitados['A','B','C']=true	1	Explora la rama a través de C.
6	C llama a vecino D (DESTINO)	[A, B, C, D]	visitados['A','B','C','D']=true	2	Caso base. Incrementa total <- 1+1=2.
7	Termina D, regresa a C	[A, B, C]	visitados['D']=false	2	**BACKTRACKING:** Desmarca D.
8	Termina C, regresa a B	[A, B]	visitados['C']=false	2	**BACKTRACKING:** Desmarca C.
9	Termina B, regresa a A	[A]	visitados['B']=false	2	**BACKTRACKING:** B agota vecinos y se desmarca.
10	A llama a vecino C	[A, C]	visitados['A','C']=true	2	A explora su segunda opción principal.
11	C llama a vecino D (DESTINO)	[A, C, D]	visitados['A','C','D']=true	3	Caso base. Incrementa total <- 2+1=3.
12	Finaliza exploración	[]	Todos en false	**3**	Retorna resultado total = 3 al wrapper.

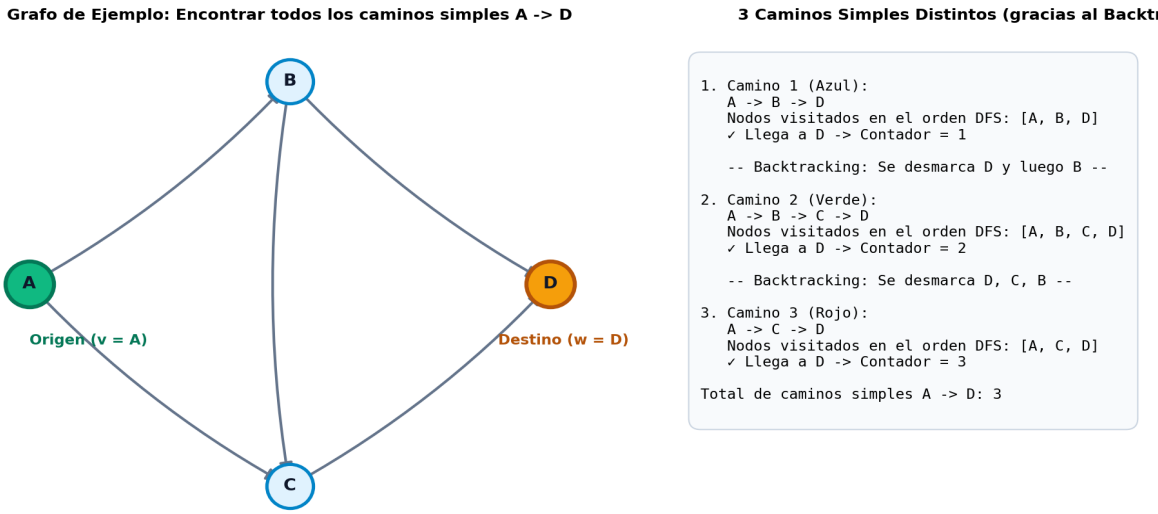


Figura 3: Árbol de búsqueda y registro de los 3 caminos simples descubiertos entre A y D.

Módulo 4: Ejercicio 1.1 del Parcial IV (caminoMaximaFelicidad)

El primer problema del Parcial IV solicita calcular la cantidad **máxima de felicidad** acumulada en un camino simple entre la isla *v* y la isla *w*. Aprovechando que el enunciado aclara que la complejidad no es restrictiva, la solución óptima en diseño es extender la plantilla de Backtracking vista en el Módulo 3. En lugar de contar rutas, el acumulador transporta la suma del costo/felicidad de las aristas recorridas, y en el caso base se conserva el valor máximo alcanzado.

```
func caminoMaximaFelicidad(g: Grafo<entero>, v: entero, w: entero) : real
Var
    visitados: Array [1..N] of bool
    maxFelicidad: real
    i: entero
Begin
    for i <- 1 to g.getNVertices() do
        visitados[i] <- false
    EFor

    visitados[v] <- true
    maxFelicidad <- -1.0  // Inicialización con valor mínimo

    maxFelicidadDFSProc(g, v, w, 0.0, visitados, maxFelicidad)

    return maxFelicidad
FFunc
proc maxFelicidadDFSProc(g: Grafo<entero>, actual: entero, destino: entero,
                         acumulado: real, ref visitados: Array of bool, ref maxFel: real)
Var
    sucesores: Lista<entero>
    u: entero
    pesoArco: real
Begin
    if actual = destino then
        if acumulado > maxFel then
            maxFel <- acumulado  // Se actualiza el récord global de felicidad
        endif
    else
        sucesores <- g.getSucesores(actual)
        while !sucesores.esVacia() do
            u <- sucesores.consultar(1)

            if !visitados[u] then
                visitados[u] <- true
                pesoArco <- g.getCostoArco(actual, u)

                maxFelicidadDFSProc(g, u, destino, acumulado + pesoArco, visitados, maxFel)

                // BACKTRACKING: Desmarcar nodo para evaluar otros caminos
                visitados[u] <- false
            endif

            sucesores.eliminar(1)
        EndWhile
    endif
FProc
```

Corrida en Frío Guiada de Maximización de Felicidad

Consideremos el grafo de islas con pesos: $v \to A$ *(+5.0)*, $v \to B$ *(+2.0)*, $A \to w$ *(+4.0)*, $B \to A$ *(+3.0)* y $B \to w$ *(+11.0)*:

Ruta Explorada	Secuencia de Vértices	Cálculo del Acumulado	Evaluación en Caso Base	Estado maxFel (ref)
Ruta 1	v -> A -> w	5.0 + 4.0 = 9.0	9.0 > -1.0 (Verdadero)	**9.0**
Ruta 2	v -> B -> A -> w	2.0 + 3.0 + 4.0 = 9.0	9.0 > 9.0 (Falso)	9.0

Ruta 3	v -> B -> w	2.0 + 11.0 = 13.0	13.0 > 9.0 (Verdadero - ¡Nuevo Máximo!)	**13.0**

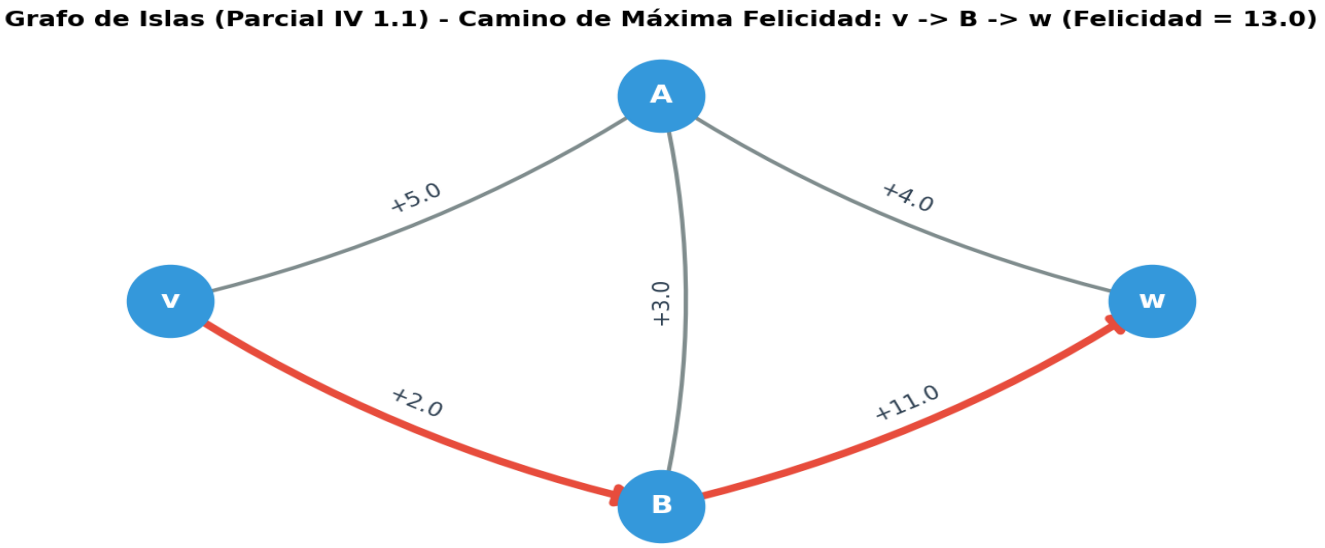


Figura 4: Grafo de islas resaltando en rojo la ruta v -> B -> w que proporciona la máxima felicidad acumulada (13.0).

Módulo 5: Tabla Resumen de Complejidades y Estructuras

Algoritmo / Operación	Tipo	Complejidad Peor Caso	Estructura Auxiliar Clave
getSucesores(g, v)	func	O(|V| + grado■(v))	Lista enlazada de arcos salientes
getPredecesores(g, v)	func	O(|V| + |E|)	Barrido completo de lista de vértices
eliminarVerticeSeguro(g, v)	proc	O(|V| + |E|)	Listas temporales de predecesores/sucesores
contarCaminosSimples(g, v, w)	func + proc	O(|V|!) [Exponencial]	Arreglo visitados + Pila de llamadas (ref)
caminoMaximaFelicidad(g, v, w)	func + proc	O(|V|!) [Exponencial]	Arreglo visitados + Acumulador maxFel (ref)